

UNIT 4

4.1 Concept and Need of Function

Function:

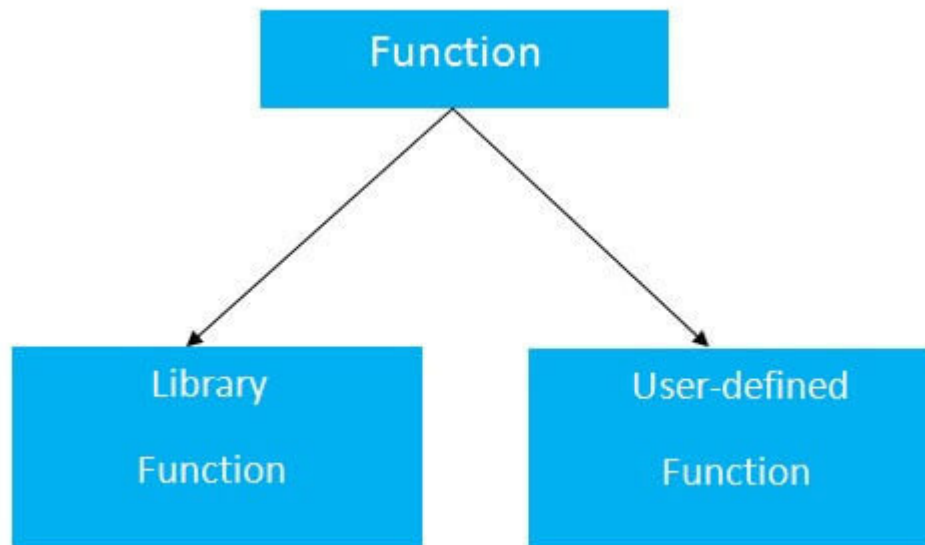
- A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function.
- Functions are used to perform certain actions and they are important for reusing code: Define the code once, and use it many times.
- A **function in C** is a set of statements that when called perform some specific task.
- It is the basic building block of a C program that provides modularity and code reusability.
- The programming statements of a function are enclosed within **{ } braces**, having certain meanings and performing certain operations.

Need of function: Once a function is defined, it can be used over and over and over again. You can invoke the same function many times in your program, which saves you work.

Types of Functions

There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as `scanf()`, `printf()`, `gets()`, `puts()`, `ceil()`, `floor()` etc.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.



C Library Function:

- The Standard Function Library in C is a huge library of sub-libraries, each of which contains the code for several functions.
- In order to make use of these libraries, link each library in the broader library through the use of header files.
- The definitions of these functions are present in their respective header files.
- In order to use these functions, we have to include the header file in the program.

Sr No.	Header Files	Description
1	<assert.h>	It checks the value of an expression that we expect to be true under normal circumstances. If the expression is a nonzero value, the assert macro does nothing.
2	<complex.h>	A set of functions for manipulating complex numbers.
3	<float.h>	Defines macro constants specifying the implementation-specific properties of the floating-point library.

Sr No.	Header Files	Description
4	<limits.h>	These limits specify that a variable cannot store any value beyond these limits, for example- An unsigned character can store up to a maximum value of 255.
5	<math.h>	The math.h header defines various mathematical functions and one macro. All the Functions in this library take double as an argument and return double as the result.
6	<stdio.h>	The stdio.h header defines three variable types, several macros, and various function for performing input and output.
7	<time.h>	Defines date and time handling functions.
8	<string.h>	Strings are defined as an array of characters. The difference between a character array and a string is that a string is terminated with a special character '\0'.

1. **stdio.h**: This library is use to use the printf() function, the header file <stdio.h> should be included in the program.

Example:

```
#include <stdio.h>
// Driver code
int main()
{
    printf("CWIT");
    return 0;
}
```

Output:

CWIT

2. **math.h**– To perform any operation related to mathematics, it is necessary to include math.h header file.

Example 1: sqrt()

Syntax-

```
double sqrt(double x)
```

```
// C program to implement the above approach
```

```
#include <math.h>
```

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
double number, squareRoot;
```

```
number = 12.5;
```

```
// Computing the square root
```

```
squareRoot = sqrt(number);
```

```
printf("Square root of %.2lf = %.2lf",
```

```
number, squareRoot);
```

```
return 0;
```

```
}
```

Output:

```
Square root of 12.50 = 3.54
```

Example 2- pow():

```
doublepow(double x, double y)
```

```
// C program to implement the above approach
```

```
#include <math.h>
```

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    double base, power, result;
```

```
    base = 10.0;
```

```
    power = 2.0;
```

```
    // Calculate the result
```

```
    result = pow(base, power);
```

```
    printf("%.1lf^%.1lf = %.2lf",  
           base, power, result);
```

```
    return 0;
```

```
}
```

Output:

```
10.0^2.0 = 100.00
```

3. **string.h**: For using string functions, it is necessary to include string.h header file in the program.

Example

1. **strcat()**: In C programming, the strcat() functions are used to concatenate(join) two strings. This function concatenates the destination string and the source string, and the result is stored in the destination string.

Syntax-

```
strcat(str1,str2);
```

2. **strcmp()**: It compares two strings. If the return value is 0 then the strings are equal or if the return value is non-zero then the strings are not equal.

Syntax:

```
strcmp(str1,str2);
```

3. **strcpy()**: The strcpy() function copies the string pointed by the source to the destination.

Syntax:

```
strcpy(str2,str1);
```

4. **strlen()**: This function calculates the length of the given string.

Syntax:

```
intstrlen(char a[]);
```

Miscellaneous Function in C:

In C programming, miscellaneous directives are used to give special instructions to the compiler. These directives are not part of the C language but are used by the preprocessor to provide useful features like conditional compilation and line control.

getchar() & putchar() function:

C getchar is a standard library function that takes a single input character from standard input. The major difference between getchar and getc is that getc can take input from any no of input streams but getchar can take input from a single standard input stream.

- It is defined inside the **<stdio.h> header file.**
- Just like getchar, there is also a function called putchar that prints only one character to the standard output stream.

Syntax: int getchar(void)

putchar(): putchar() function puts the passed character on the screen and returns the ASCII value of the character. This function puts only single character at a time.

Example:

```
#include<stdio.h>

#include<conio.h>

void main()

{

char c;

printf("\n Enter a character \n");

c=getchar();//get a single character

printf("You have passed");

putchar(c);// print a single character using putchar

getch();

}
```

Output: Enter a character

A

You have passed A

malloc() & calloc() function in C:

1. malloc():

- The “**malloc**” or “**memory allocation**” method in C is used to dynamically allocate a single large block of memory with the specified size.
- It returns a pointer of type void which can be cast into a pointer of any form. It doesn't initialize memory at execution time so that it has initialized each block with the default garbage value initially.

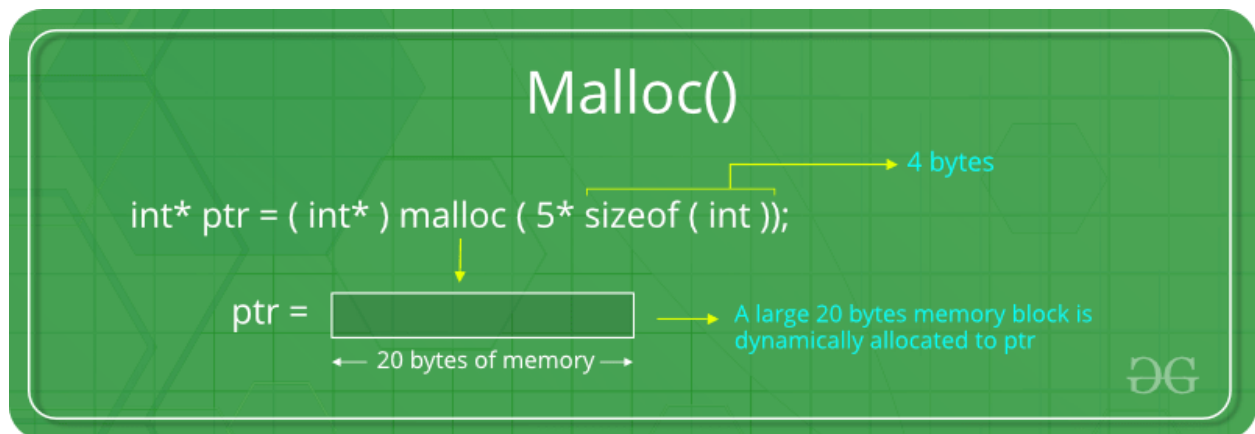
Syntax of malloc() in C

```
ptr = (cast-type*) malloc(byte-size)
```

Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.



```
#include <stdio.h>
#include <stdlib.h>
int main()
{
```

```
    // This pointer will hold the
    // base address of the block created
    int* ptr;
    int n, i;
```

```

// Get the number of elements for the array
printf("Enter number of elements:");
scanf("%d",&n);
printf("Entered number of elements: %d\n", n);

// Dynamically allocate memory using malloc()
ptr = (int*)malloc(n * sizeof(int));

// Check if the memory has been successfully
// allocated by malloc or not
if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {

    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc.\n");

    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}

```

2. calloc():

- “**calloc**” or “**contiguous allocation**” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:
- It initializes each block with a default value ‘0’.
- It has two parameters or arguments as compare to malloc().

Syntax of calloc() in C:

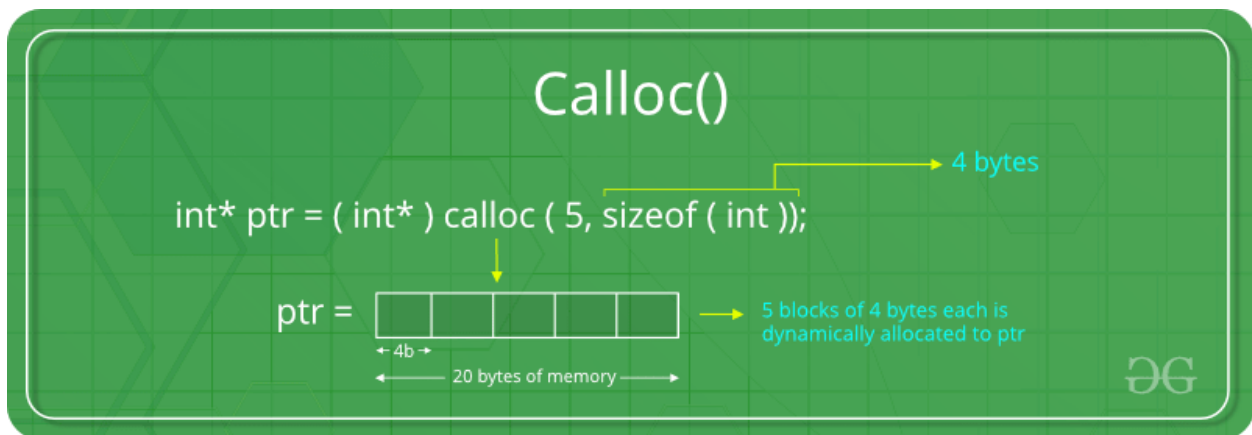
```
ptr = (cast-type*)calloc(n, element-size);
```

here, n is the no. of elements and element-size is the size of each element.

For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float.



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    // This pointer will hold the base address of the block created
```

```
    int* ptr;
```

```
    int n, i;
```

```
    // Get the number of elements for the array
```

```
    n = 5;
```

```
    printf("Enter number of elements: %d\n", n);
```

```

// Dynamically allocate memory using calloc()
ptr = (int*)calloc(n, sizeof(int));

// Check if the memory has been successfully
// allocated by calloc or not
if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {
    // Memory has been successfully allocated
    printf("Memory successfully allocated using calloc.\n");

    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}

```

4.3 User defined Function in C:

- A **user-defined function** is a type of function in C language that is defined by the user himself to perform some specific task.
- It provides code reusability and modularity to our program.
- User-defined functions are different from built-in functions as their working is specified by the user and no header file is required for their usage.
- The programming statements of a function are enclosed within **{ } braces**, having certain meanings and performing certain operations.

Syntax of Functions in C:

The syntax of function can be divided into 3 aspects:

1. **Function Declaration**
2. **Function Definition**
3. **Function Calls**

1. Function Declarations

- In a function declaration, we must provide the function name, its return type, and the number and type of its parameters.
- A function declaration tells the compiler that there is a function with the given name defined somewhere else in the program.

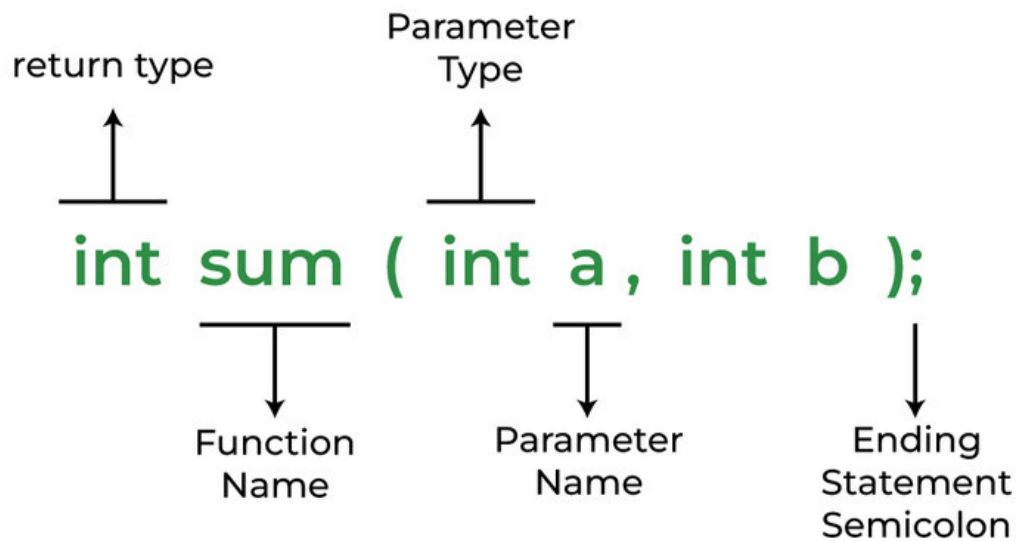
Syntax:

```
return_typename_of_the_function(parameter_1, parameter_2);
```

The parameter name is not mandatory while declaring functions. We can also declare the function without using the name of the data variables.

Example:

```
intsum(inta, intb);  
intsum(int , int);
```



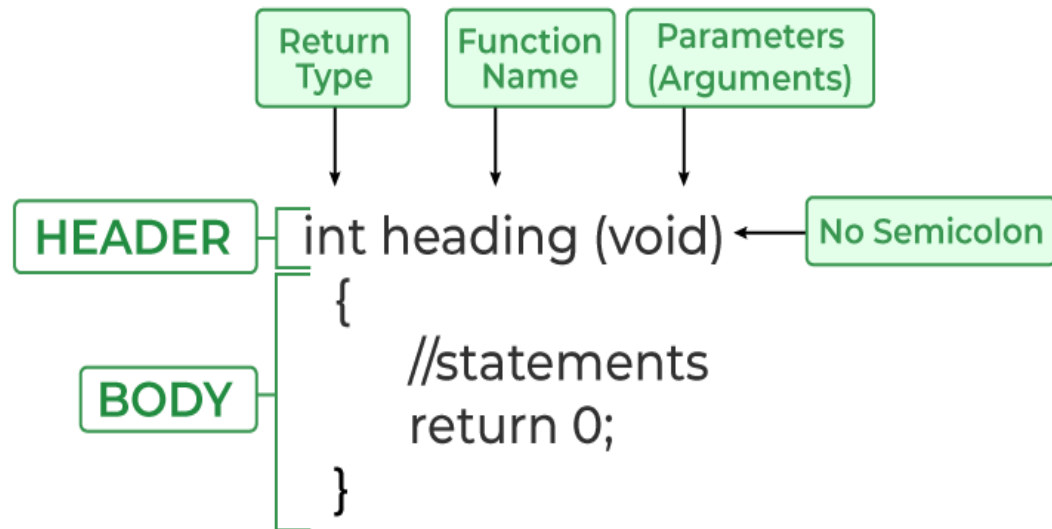
Note: A function in C must always be declared globally before calling it.

2. Function Definition

- The function definition consists of actual statements which are executed when the function is called (i.e. when the program control comes to the function).
- A C function is generally defined and declared in a single step because the function definition always starts with the function declaration so we do not need to declare it explicitly.
- The below example serves as both a function definition and a declaration.

```
return_type function_name (para1_type para1_name, para2_type para2_name)  
{  
    // body of the function  
}
```

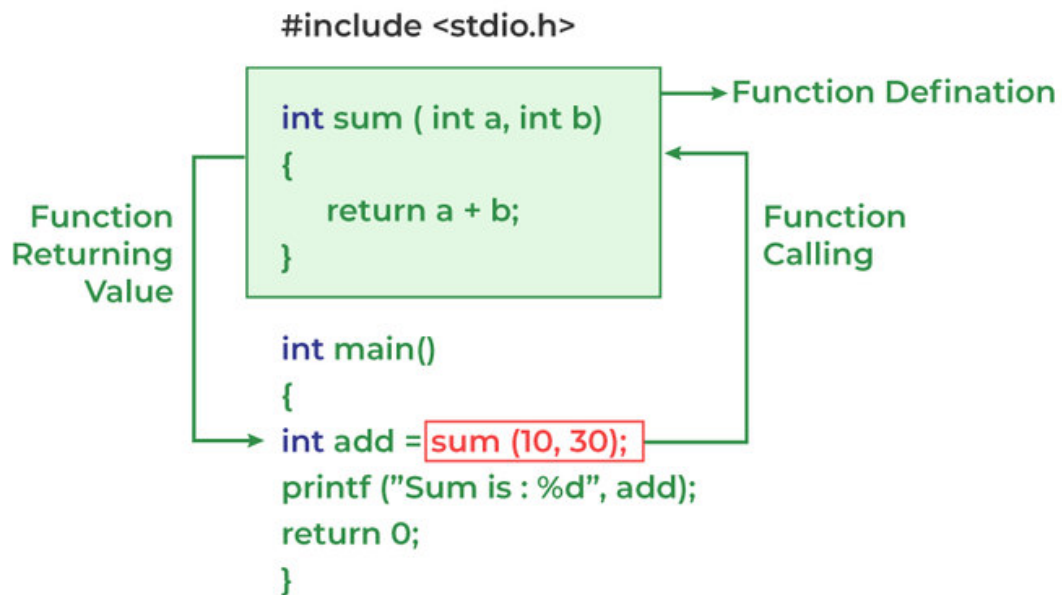
Function Definition



3. Function Call

- A function call is a statement that instructs the compiler to execute the function. We use the function name and parameters in the function call.
- In the below example, the first sum function is called and 10,30 are passed to the sum function. After the function call sum of a and b is returned and control is also returned back to the main function of the program.

Working of Function in C



Note: Function call is necessary to bring the program control to the function definition. If not called, the function statements will not be executed.

Example of C Function

// C program to show function call and definition

```
#include <stdio.h>
```

// Function that takes two parameters a and b as inputs and returns their sum

```
int sum(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
int main()
```

```
{
```

```
// Calling sum function and
```

```
// storing its value in add variable
```

```
int add = sum(10, 30);  
printf("Sum is: %d", add);  
return 0;  
}
```

Output

```
Sum is: 40
```

Scope of variable in C

The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable and it is treated as an undeclared identifier.

- The scope is the area under which a variable is visible.
- The scope of an identifier is the part of the program where the identifier may directly be accessible.
- We can only refer to a variable in its scope.

C program to illustrate the scope of a variable

```
#include <stdio.h>  
  
int main()  
{  
    // Scope of this variable is within main() function only.  
    int var = 34;  
    printf("%d", var);  
    return 0;  
}  
  
// function where we try to access the var defined in main()  
void func() { printf("%d", var);  
}
```

Output:

```
solution.c: In function 'func':  
solution.c:15:28: error: 'var' undeclared (first use in this function)  
voidfunc() { printf("%d", var); }
```

Types of Scope Rules in C:

1. **Global Scope**
2. **Local Scope**

1. Global Scope in C

The global scope refers to the region outside any block or function.

- The variables declared in the global scope are called **global variables**.
- Global variables are visible in **every part of the program**.
- Global is also called **File Scope** as the scope of an identifier starts at the beginning of the file and ends at the end of the file.

Example

// C program to illustrate the global scope

```
#include <stdio.h>
```

```
// variable declared in global scope
```

```
int global = 5;
```

```
// global variable accessed from
```

```
// within a function
```

```
void display()
```

```
{
```

```
    printf("%d\n", global);
```

```
}
```

```
// main function
int main()
{
    printf("Before change within main: ");
    display();

    // changing value of global
    // variable from main function
    printf("After change within main: ");
    global = 10;
    display();
}
```

Output:

```
Before change within main: 5
After change within main: 10
```

2. Local Scope in C:

The local scope refers to the region inside a block or a function. It is the space enclosed between the **{ } braces**.

- The variables declared within the local scope are called **local variables**.
- Local variables are visible in the **block they are declared** in and **other blocks nested** inside that block.
- Local scope is also called **Block scope**.
- Local variables have internal linkage.

Example:

```
// C program to illustrate the local scope
#include <stdio.h>
```

```

// Driver Code
int main()
{
    {
        int x = 10, y = 20;
        {
            // The outer block contains
            // declaration of x and
            // y, so following statement
            // is valid and prints
            // 10 and 20
            printf("x = %d, y = %d\n", x, y);
            {
                // y is declared again,
                // so outer block y is
                // not accessible in this block
                int y = 40;

                // Changes the outer block
                // variable x to 11
                x++;

                // Changes this block's
                // variable y to 41
                y++;
            }
        }
    }
}

```

```

        printf("x = %d, y = %d\n", x, y);
    }

    // This statement accesses only outer block's variables
    printf("x = %d, y = %d\n", x, y);
}

}

return 0;
}

```

Output

```

x = 10, y = 20
x = 11, y = 41
x = 11, y = 20

```

4.4 Function Parameters:

- Information can be passed to function as a parameter. Parameter acts as variable inside the function.
- Parameters are specified after the function name, inside the parantheses.
- You can add many parameters as you want just separate them with comma.

Syntax:

```

Returntypefunction_name (parameter 1, parameter 2)
{
    //code to be executed
}

```

Function parameter passing:

- A function in C can be called either with arguments or without arguments. These functions may or may not return values to the calling functions.

1. Call by Value

- Call by value in C is where in the arguments we pass value and that value can be used in function for performing the operation.
- Values passed in the function are stored in temporary memory so the changes performed in the function don't affect the actual value of the variable passed.

Example:

// C Program to implement Call by value

```
#include <stdio.h>
```

```
// Call by value
```

```
int sum(int x, int y)
```

```
{
```

```
    int c;
```

```
    c = x + y;
```

```
    // Integer value returned
```

```
    return c;
```

```
}
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
    // Integer Declared
```

```
    int a = 3, b = 2;
```

```
// Function Called  
  
int c = sum(a, b);  
  
printf("Sum of %d and %d : %d", a, b, c);  
  
return 0;  
  
}
```

Output

```
Sum of 3 and 2 : 5
```

2. Call by Reference

- Call by reference is the method in C where we call the function with the passing address as arguments.
- We pass the address of the memory blocks which can be further stored in a pointer variable that can be used in the function.
- Now, changes performed in the values inside the function can be directly reflected in the main memory.

Example:

```
// C Program to implement
```

```
// Call by reference
```

```
#include <stdio.h>
```

```
// Call by reference
```

```
void swap(int* x, int* y)
```

```
{
```

```
    int temp = *x;
```

```
    *x = *y;
```

```
    *y = temp;
```

```
}
```

```
// Driver Code

int main()
{
    // Declaring Integer

    int x = 1, y = 5;

    printf("Before Swapping: x:%d , y:%d\n", x, y);


    // Calling the function

    swap(&x, &y);

    printf("After Swapping: x:%d , y:%d\n", x, y);


    return 0;
}
```

Output

Before Swapping: x:1 , y:5

After Swapping: x:5 , y:1

Types of Function According to Arguments and Return Value

Functions can be differentiated into 4 types according to the arguments passed and value returns these are:

1. Function with arguments and return value
2. Function with arguments and no return value
3. Function with no arguments and with return value
4. Function with no arguments and no return value

1. Example for Function with argument and with return value

Example 1:

```
#include<stdio.h>
int sum(int, int);
void main()
{
int a,b,result;
printf("\nGoing to calculate the sum of two numbers:");
printf("\nEnter two numbers:");
scanf("%d %d",&a,&b);
result = sum(a,b);
printf("\nThe sum is : %d",result);
}
int sum(int a, int b)
{
return a+b;
}
```

Output

```
Going to calculate the sum of two numbers:
Enter two numbers:10
20
The sum is : 30
```

Example 2: Program to check whether a number is even or odd

```
#include<stdio.h>
int even_odd(int);
void main()
{
int n,flag=0;
printf("\nGoing to check whether a number is even or odd");
printf("\nEnter the number: ");
scanf("%d",&n);
```

```

flag = even_odd(n);
if(flag == 0)
{
    printf("\nThe number is odd");
}
else
{
    printf("\nThe number is even");
}
}
int even_odd(int n)
{
    if(n%2 == 0)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

```

Output:

```

Going to check whether a number is even or odd
Enter the number: 100
The number is even

```

2. Example for Function with argument and without return value

Example 1

```

#include<stdio.h>
void sum(int, int);
void main()

```

```

{
    int a,b,result;
    printf("\nGoing to calculate the sum of two numbers:");
    printf("\nEnter two numbers:");
    scanf("%d %d",&a,&b);
    sum(a,b);
}
void sum(int a, int b)
{
    printf("\nThe sum is %d",a+b);
}

```

Output:

Going to calculate the sum of two numbers:

Enter two numbers 10
24

The sum is 34

Example 2: program to calculate the average of five numbers.

```

#include<stdio.h>
void average(int, int, int, int, int);
void main()
{
    int a,b,c,d,e;
    printf("\nGoing to calculate the average of five numbers:");
    printf("\nEnter five numbers:");
    scanf("%d %d %d %d %d",&a,&b,&c,&d,&e);
    average(a,b,c,d,e);
}
void average(int a, int b, int c, int d, int e)
{
    float avg;

```

```
avg = (a+b+c+d+e)/5;
printf("The average of given five numbers : %f",avg);
}
```

Output

```
Going to calculate the average of five numbers:
Enter five numbers:10
20
30
40
50
The average of given five numbers : 30.000000
```

3. Example for Function without argument and with return value

Example 1

```
#include<stdio.h>
int sum();
void main()
{
    int result;
    printf("\nGoing to calculate the sum of two numbers:");
    result = sum();
    printf("%d",result);
}
int sum()
{
    int a,b;
    printf("\nEnter two numbers");
    scanf("%d %d",&a,&b);
    return a+b;
}
```

Output:

Going to calculate the sum of two numbers:

Enter two numbers 10
24

The sum is 34

Example 2: program to calculate the area of the square

```
#include<stdio.h>
int sum();
void main()
{
    printf("Going to calculate the area of the square\n");
    float area = square();
    printf("The area of the square: %f\n",area);
}
int square()
{
    float side;
    printf("Enter the length of the side in meters: ");
    scanf("%f",&side);
    return side * side;
}
```

Output

```
Going to calculate the area of the square
Enter the length of the side in meters: 10
The area of the square: 100.000000
```

4. Example for Function without argument and return value

```
#include<stdio.h>
void sum();
void main()
```

```

{
    printf("\nGoing to calculate the sum of two numbers:");
    sum();
}
void sum()
{
    int a,b;
    printf("\nEnter two numbers");
    scanf("%d %d",&a,&b);
    printf("The sum is %d",a+b);
}

```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

Return Value:

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Example without return value:

```

void hello(){
    printf("hello c");
}

```

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Example with return value:

```
int get(){
    return 10;
}
```

C return statement:

- C return statement ends the execution a function of and returns the control to the function from where it was called.
- The return statement may or may not return a values depending upon the return type of the function.
- For example: int returns an integer values, void returns nothing,etc.

Syntax: return return_value;

Advantage of functions in C:

There are the following advantages of C functions.

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
-

4.5 Recursion in C

Recursion is the process of a function calling itself repeatedly till the given condition is satisfied. A function that calls itself directly or indirectly is called a recursive function and such kind of function calls are called recursive calls.

Basic Structure of Recursive Functions

The basic syntax structure of the recursive functions is:

```
typefunction_name (args) {
// function statements
    // base condition
    // recursion case (recursive call)
}
```

```
// C Program to calculate the sum of first N natural numbers

// using recursion

#include <stdio.h>

intnSum(int n)
{
    // base condition to terminate the recursion when N = 0
    if (n == 0) {
        return 0;
    }

    // recursive case / recursive call
    int res = n + nSum(n - 1);
    return res;
}

int main()
{
    int n = 5;

    // calling the function
    int sum = nSum(n);

    printf("Sum of First %d Natural Numbers: %d", n, sum);

    return 0;
}
```

Output

Sum of First 5 Natural Numbers: 15

Fundamentals of C Recursion

The fundamental of recursion consists of two objects which are essential for any recursive function. These are:

1. Recursion Case
2. Base Condition

```
int nSum(int n)
{
    if (n==0) {
        return 0;
    }
```

Base
condition

```
int res = n+ nsum(n-1);
```

```
return res;
}
```

Recursive
case

1. Recursion Case

- The recursion case refers to the recursive call present in the recursive function. It decides what type of recursion will occur and how the problem will be divided into smaller subproblems.
- The **recursion case defined in the nSum()** function of the above example is:

```
int res = n + nSum(n - 1);
```
- The recursion case is often represented mathematically as a **recurrence relation**. For the above case:
$$f(N) = N + f(N - 1);$$

2. Base Condition

- The base condition specifies when the recursion is going to terminate. It is the condition that determines the exit point of the recursion.
- **Note:** It is important to define the base condition before the recursive case otherwise, the base condition may never be encountered and recursion might continue till infinity.

```
if(n == 0) {  
    return 0;  
}
```
- Now that the basic terminologies and fundamentals are out of the way, let's move on to understand how the recursion works in C.

How Recursion works in C?

```
int res = n + nSum(n - 1);
```

In the example, $n = 5$, so as **nSum(5)**'s recursive case, we get

```
int res = 5 + nSum(4);
```

In **nSum(4)**, the recursion case and everything else will be the same, but $n = 4$. Let's evaluate the recursive case for $n = 4$,

```
int res = 4 + nSum(3);
```

Similarly, for **nSum(3)**, **nSum(2)** and **nSum(1)**

```
int res = 3 + nSum(2); // nSum(3)
```

```
int res = 2 + nSum(1); // nSum(2)
```

```
int res = 1 + nSum(0); // nSum(1)
```

Now recall that the **return value** of the **nSum()** function is this same integer named **res**. So, instead of the function, we can put the value returned by these functions. As such, for **nSum(5)**, we get

```
int res = 5 + 4 + nSum(3);
```

Similarly, putting return values of nSum() for every n, we get
`int res = 5 + 4 + 3 + 2 + 1 + nSum(0);`

In nSum() function, the **base condition** is

```
if (n == 0) {  
    return 0;  
}
```

which means that when nSum(0) will return 0. Putting this value in nSum(5)'s recursive case, we get

```
int res = 5 + 4 + 3 + 2 + 1 + 0 = 15
```

Advantages of C Recursion

The advantages of using recursive methods over other methods are:

1. Recursion can effectively reduce the length of the code.
2. Some problems are easily solved by using recursion like the tower of Hanoi and tree traversals.
3. Data structures like linked lists, trees, etc. are recursive by nature so recursive methods are easier to implement for these data structures.

Disadvantages of C Recursion

As with almost anything in the world, recursion also comes with certain limitations some of which are:

1. Recursive functions make our program a bit slower due to function call overhead.
2. Recursion functions always take extra space in the function call stack due to separate stack frames.
3. Recursion methods are difficult to understand and implement.